

Computer Organization and Architecture		
Lab #:	08	
Lab Title:	Half adder, Full Adder	
Submitted by:		
Names		Registration #
Sumaira Safeer		FA22-BCE-019

Rubrics name & number					Marks	
					In-Lab	Post-Lab
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.					
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.					
Design	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.					
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.					
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.					
	R13: Management of Team Work: Ability to appreciate, understand and work with multidisciplinary team members.					
Rubrics #	R2	R6	R8	R9	R12	R13
In Lab						
Post- Lab						

Introduction to Verilog HDL Combinational Circuits1

Task: Implement the Half adder:

Objective:

The objective of this task is to design and implement the Half Adder circuits to perform binary addition.

Half Adder: Understand and verify the working of a basic half adder that adds two single-bit binary numbers and provides two outputs: **Sum** and **Carry**.

Verilog Code:

```
module my_half_adder(sum, c_out, a, b);  
    output sum, c_out;  
    input a,b;  
    xor M1(sum, a, b);    //Instantiating primitive XOR  
    and M2(c_out, a, b);   //Instantiating primitive AND  
    //Any instance name can be //given. M1 and M2 are not //keywords. You may use your  
    //name  
endmodule
```

Testbench:

```
module test_my_half_adder;  
    wire sum, c_out; //Outputs of UUT (in this case  
    // 'module my_half_adder') declared as //wire  
    reg a, b; // Inputs of UUT declared as 'reg'  
    my_half_adder AA(sum, c_out, a, b);  
  
    initial begin                //Initially a=x, b=x  
        #10 a = 0; b = 0;        //After 10ns a=0, b=0  
        #10      b = 1;          //After 20ns a=0, b=1
```

```
#10 a = 1;                //After 30ns a=1, b=1
#10          b = 0;        //After 40ns a=1, b=0
end
endmodule
```

Result:



Task: 02 Full Adder:

Objectives:

- Gate Level Modeling.

About Verilog:

Verilog HDL is a programming language used to describe a digital system: for example, a network switch, a microprocessor or a memory or a simple flip-flop. This just means that, by using a HDL, one can describe any (digital) hardware at any level.

Full Adder Verilog:

```
module my_full_adder(input [2:0] x, output [1:0] y);
//Different style + use of vectors
```

```
// x is 3-bit input & y is 2-bit output ,sum and carry;
wire sum1,carry1,carry2;
my_half_adder H1(sum1, carry1, x[0], x[1]);
my_half_adder H2(y[0], carry2, sum1, x[3]);
or (y[1],carry1,carry2);
endmodule
```

TestBench:

```
module tb_my_full_adder;
reg [2:0] x;    // Inputs of UUT decided as ?reg?
wire [1:0] y;    //Outputs of UUT (is this case module verilog?) decided as // wire

my_full_adder UUT ( x(x), y(y));
initial begin
$display("x (input) | y (output: sum, carry)"); // Display header
$display(" ");

x = 3'b000;    //Even”or “Odd
#10;
x = 3'b001;
#10;
x = 3'b010;
#10;
x = 3'b011;
#10;
x = 3'b100;
#10;
x = 3'b101;
#10;
x = 3'b110;
```

```

#10;
x = 3'b111;
#10;
$finish;    // End simulation
end
// Display outputs on change
always @(x or y) begin
$display("%b | %b", x, y);
end
endmodule

```

Result:

